

CAMPUS EATS

Process Document

Version 2.3 | Online REST API Edition | Full RBAC on Web and Mobile

Prepared for Rosebank International South Africa

Contents

1. INTRODUCTION.....	4
1.1 Purpose of the System.....	4
1.2 Problem Statement.....	4
1.3 Proposed Solution.....	4
2. ORGANIZATIONAL CONTEXT.....	6
2.1 Organization Overview.....	6
2.2 Business Objectives.....	6
3. STAKEHOLDER IDENTIFICATION.....	7
3.1 Primary Stakeholders.....	7
3.2 Secondary Stakeholders.....	7
4. SYSTEM SCOPE AND BOUNDARIES.....	8
4.1 In Scope.....	8
5. BUSINESS PROCESS OVERVIEW.....	9
5.1 Order Placement Process.....	9
5.2 Vendor Order Management Process.....	9
6. FUNCTIONAL REQUIREMENTS.....	10
6.1 Student and Standard User Functional Requirements.....	10
6.2 Vendor Functional Requirements.....	10
6.3 Administrator Functional Requirements.....	10
7. NON-FUNCTIONAL REQUIREMENTS.....	12
8. SYSTEM ARCHITECTURE.....	13
8.1 High-Level Architecture.....	13
8.2 Core System Modules.....	13
9. DATABASE DESIGN OVERVIEW.....	14
9.1 Core Entities.....	14
9.2 Key Relationships.....	14
10. USER INTERFACE DESIGN CONSIDERATIONS.....	15
10.1 Student and Standard User Screens.....	15
10.2 Vendor Screens.....	15
10.3 Administrator Screens.....	15
11. MOBILE USER ID SYSTEM.....	16

11.1 16-Character Global Identifier.....	16
11.2 Order ID System.....	16
12. USER ROLES (4 TYPES, 8 DEMO ACCOUNTS).....	17
12.1 Student Role.....	17
12.2 Standard Role.....	17
12.3 Vendor Role.....	17
12.4 Admin Role.....	17
13. ACCOUNT CREATION PROCESS.....	18
Step 1: User Registration.....	18
Step 2: Storage.....	18
Step 3: Output to User.....	18
14. ACCOUNT RECOVERY SYSTEM.....	19
15. LOGIN SYSTEM.....	20
16. SESSION MANAGEMENT.....	21
17. SECURITY IMPLEMENTATION.....	22
18. ORDER FLOW (END-TO-END).....	23
19. ONLINE-FIRST BEHAVIOR.....	24
Data Access.....	24
Network Dependency.....	24
Real-Time Updates.....	24
20. SYSTEM INTEGRITY RULES.....	25
21. FEASIBILITY ANALYSIS.....	26
21.1 Technical Feasibility.....	26
21.2 Economic Feasibility.....	26
21.3 Operational Feasibility.....	26
22. CONCLUSION.....	27
23. References.....	28

1. INTRODUCTION

1.1 Purpose of the System

Campus Eats is a food ordering and pickup platform built for Rosebank International South Africa. Customers order from registered campus vendors, vendors manage their menus and incoming orders, and administrators oversee the whole system.

There are two client applications, and both support all four roles: Student, Standard, Vendor, and Admin.

1. An Android app, usable by any role. A customer places orders from it, but a vendor can also log in and manage orders from their phone, and an admin can check reports on the go.
2. A PHP web application, also usable by any role. A vendor or admin will likely prefer the larger screen for day-to-day management, but a student or standard user can just as easily log in and order from a browser.

Which app someone uses is a matter of convenience, not a restriction tied to their role. Role-based access control is enforced by the backend API, not by the client they happen to open.

1.2 Problem Statement

Ordering food at a physical stall on campus runs into the same problems most manual processes do:

1. Long queues during peak hours.
2. Miscommunication between students and vendors about what was actually ordered.
3. No digital record of orders.
4. Payments handled by hand.
5. No transaction history to check against.
6. Poor coordination between vendors and whoever is handling pickup.
7. Nothing tying ordering, payment, vendor management, and delivery together.

1.3 Proposed Solution

Campus Eats addresses this with a REST API that acts as the single source of truth for the system. Both the Android app and the PHP web app talk to the same backend and expose the same functionality across all four roles, so a user's experience depends on their role and permissions, not on which app they happen to open.

The solution covers:

1. A customer interface for browsing menus and placing orders.

2. A vendor portal for managing menu items and processing orders.
3. An administrative dashboard for system control and reporting.

All three of the above are available on both the Android app and the PHP website, gated by the user's role rather than the platform.

2. ORGANIZATIONAL CONTEXT

2.1 Organization Overview

The system is meant for a college campus with several food vendors, a base of registered customers (students and standard users), and administrative staff. Any of these groups can work through either the Android app or the web app, since both platforms carry the same role-based access model.

2.2 Business Objectives

The college wants to:

- Improve takeaway service for customers.
- Increase vendor operational efficiency.
- Improve revenue tracking and reporting.
- Reduce physical congestion at vendor stalls.
- Provide management with structured data insights.

3. STAKEHOLDER IDENTIFICATION

3.1 Primary Stakeholders

1. Students: browse menus, place orders, and pay, through either the Android app or the website. They get student pricing.
2. Standard users: same flow on either platform, without the student discount.
3. Vendors: campus food stalls and cafeterias that prepare and fulfill orders, and manage listings and inventory, through either the Android app or the website.
4. System administrator: manages users, vendors, reports, and configuration, through either the Android app or the website, with full system and security oversight.

3.2 Secondary Stakeholders

1. Campus management.
2. IT support department.
3. Finance department.

4. SYSTEM SCOPE AND BOUNDARIES

4.1 In Scope

1. User registration and authentication through the REST API, for all four roles, on both clients.
2. Role-based access control (RBAC) enforced by the API, consistently across both clients.
3. Menu browsing and search.
4. Cart management on either client.
5. Order placement and confirmation.
6. Order status tracking across all clients.
7. Administrative reporting, available on both the Android app and the website.
8. Online-first operation. The system needs an internet connection to send and retrieve data from the API.

5. BUSINESS PROCESS OVERVIEW

5.1 Order Placement Process

1. Customer logs into either the mobile app or the website.
2. Customer browses vendor menus, pulled live from the API.
3. Customer adds items to the cart.
4. Customer confirms the order and pays.
5. The order is sent to the API.
6. The vendor sees the new order come in; on whichever client they are using.
7. Vendor accepts or rejects it.
8. Vendor prepares the order.
9. Status is updated to "ready for pickup" through the API.

5.2 Vendor Order Management Process

1. Vendor logs into either the mobile app or the website.
2. Vendor checks pending orders, pulled live from the API.
3. Vendor accepts or rejects the order.
4. Vendor updates preparation status.
5. Vendor marks the order ready for pickup.

6. FUNCTIONAL REQUIREMENTS

Each role's functionality below is available on both the Android app and the PHP website. The API enforces what a given role can do; the client just decides how it is presented.

6.1 Student and Standard User Functional Requirements

Students and standard users should be able to, on either client:

1. Register and log in.
2. Browse and search vendor menus.
3. Add items to a cart.
4. Modify what's in the cart.
5. Place orders.
6. Track order status.
7. View order history.
8. Recover their account using the registered email.

6.2 Vendor Functional Requirements

Vendors should be able to, on either client:

1. Register a vendor profile.
2. Add new menu items.
3. Update existing menu items.
4. Remove menu items.
5. Set item availability.
6. Accept or reject orders.
7. Update order status.
8. Track inventory per item.

6.3 Administrator Functional Requirements

Administrators should be able to, on either client:

1. Manage user accounts.
2. Manage vendor accounts.
3. View system reports.
4. Monitor transactions.
5. Deactivate or suspend accounts.

6. View security logs.

7. NON-FUNCTIONAL REQUIREMENTS

1. The system must support at least 1,000 concurrent users.
2. Response time must stay under 3 seconds per request.
3. Authentication must be secure for all four roles.
4. Data must move over HTTPS.
5. The system must run on Android devices and modern web browsers, with full functional parity across roles on both.
6. Data integrity and confidentiality must be maintained.
7. The system must operate online to send and retrieve data through the REST API.

8. SYSTEM ARCHITECTURE

8.1 High-Level Architecture

The system is a distributed, RESTful setup with three layers.

Backend layer (REST API): built on ASP.NET Core 8 with Entity Framework Core. This is the central server; all business logic, data access, and RBAC enforcement happen here, regardless of which client made the request. It runs on a central server, whether that is Azure, AWS, or something on-premise.

Client layers:

- Android app: built in Kotlin with Jetpack Compose (or an XML-based UI), using Retrofit or Ktor to talk to the API. The UI adapts based on the logged-in user's role, so a vendor sees menu and order management screens, an admin sees dashboards and logs, and a student or standard user sees the ordering flow.
- TypeScript web application: built in TypeScript using HTTP client to talk to the API. Same principle: the interface shown depends on the user's role, not on an assumption baked into the platform.
- PHP web application: built in PHP using HTTP client to talk to the API. Same principle: the interface shown depends on the user's role, not on an assumption baked into the platform

Security layer: every authenticated request needs an API key (the user code). Passwords are hashed, not stored in plain text. HTTPS is mandatory between clients and the API. Role checks happen server-side, so a client cannot grant itself permissions it was not given.

8.2 Core System Modules

1. Authentication and authorization module: handles registration, login, and API key generation for all roles.
2. User management module: handles user profiles and roles.
3. Vendor management module: handles vendor profiles.
4. Menu management module: handles menu items.
5. Order processing module: handles order creation, retrieval, and management.
6. Payment management module: simulated payment processing, available on either client.
7. Reporting module: analytics and reporting tools, available on either client for roles permitted to see them.

9. DATABASE DESIGN OVERVIEW

9.1 Core Entities

Entity	Key Fields	Notes
User	userEmail, passwordHash, role, userCode	role drives access control across both clients; userCode doubles as the API key
Restaurant	restaurantName, address, type, parkingLot	represents a vendor
Menu	itemName, itemPrice, itemDescription, imageUrl	linked to a Restaurant
Order	totalPrice, userDetails	master order; can contain one or more individual order records
Order Items	orderId (foreign key)	items within a master order

9.2 Key Relationships

1. A user has one role (student, standard, vendor, or admin).
2. A user can have many orders.
3. A restaurant has many items on the menu.
4. A master order contains one or more individual orders.

10. USER INTERFACE DESIGN CONSIDERATIONS

Both the Android app and the website present role-appropriate screens. What follows is organized by role, not by platform, since each role's screens exist on both clients.

10.1 Student and Standard User Screens

- Home screen: shows all vendors and available items.
- My Orders screen: shows receipts and order history.
- Cart screen: shows everything the user wants to buy, with the fee, tax, and discount rules below.

Rule	Detail
Service fee	Under 500 rand: 10%. Between 500 and 1,000 rand: 6.5%. 1,000 rand and above: no fee.
Tax	20%
Rounding	Round up to the nearest 5 rand
Student discount	2.5% off, applied only to the Student role

- Feedback screen: lets the user submit a compliment or complaint.

10.2 Vendor Screens

- Home screen: vendor summary.
- Menu screen: CRUD operations for menu items.
- Orders screen: order management.
- Reports screen: sales analytics.

10.3 Administrator Screens

- Home screen: system statistics.
- User Management screen.
- Vendor Management screen.
- Order Management screen.
- Transactions screen.
- Reports screen.
- Security Log screen.

All interfaces follow consistent navigation and usability standards, regardless of which platform they are rendered on.

11. MOBILE USER ID SYSTEM

11.1 16-Character Global Identifier

Every user, regardless of role, gets a unique, immutable 16-character alphanumeric ID at registration.

- Format: XXXX-XXXX-XXXX-XXXX (stored as one continuous string).
- Example: A9F3-KL21-ZX90-QWER.
- Generated with a cryptographically secure random generator.
- Required for account recovery.
- Used for cross-device identity restoration, including switching between the Android app and the website.

11.2 Order ID System

Every order gets a unique ID at checkout.

- Format: ORD-YYYYMMDD-XXXXXXXXXX.
- Example: ORD-20260627-8F3A91BC.
- Globally unique and immutable once created.
- Linked to the user's ID.

12. USER ROLES (4 TYPES, 8 DEMO ACCOUNTS)

Each role below works on both the Android app and the PHP website. There is no platform restriction tied to roles.

12.1 Student Role

- Access campus vendors.
- Browse menus.
- Place orders.
- View order history.
- Receive student-specific pricing.

Demo accounts: Student_01, Student_02.

12.2 Standard Role

- Full consumer experience.
- Cart management.
- Checkout and order tracking.
- Order modification before confirmation.

Demo accounts: Standard_01, Standard_02.

12.3 Vendor Role

- Manage product listings.
- Accept or reject orders.
- Update order status: Pending → Preparing → Ready → Completed.
- Inventory tracking per item.
- View sales analytics.

Demo accounts: Vendor_01, Vendor_02.

12.4 Admin Role

- Full system control.
- Manage users and vendors.
- View system logs.
- Audit orders.
- Security monitoring dashboard.

Demo accounts: Admin_01, Admin_02.

13. ACCOUNT CREATION PROCESS

Step 1: User Registration

User enters email, password, and role. Registration works the same way whether it happens on the Android app or the website. The system generates a 16-character user ID, hashes the password with BCrypt or Argon2, and sets the account status to active by default.

Step 2: Storage

The record goes into the central database through the API, indexed by user ID and email for fast lookup.

Step 3: Output to User

The user sees their 16-character ID and is told to store it somewhere safe, since it is the key to recovering the account on either platform.

14. ACCOUNT RECOVERY SYSTEM

Recovery runs on the 16-character user ID and works the same way regardless of which client the user is on.

- 6 User selects "Recover Account".
- 7 Users enter their ID.
- 8 The system checks it against the API.
- 9 If it matches, identity is verified.
- 10 User sets a new password or confirms biometric authentication.
- 11 Password reset only happens after verification.

Security rule: the user ID alone is not sufficient to log in. It must be paired with a password, biometric authentication, or a session token.

15. LOGIN SYSTEM

Login needs the 16-character user ID and a password, on either client. The password hash comparison happens on the API server. On success, a session token is issued with an expiry attached.

16. SESSION MANAGEMENT

Sessions live on the client and are validated against the API. They expire after 15 to 60 minutes of idle time. Logging out clears the local session record, and expired tokens cannot be reused. A user's session on the Android app is independent of any session they hold on the website.

17. SECURITY IMPLEMENTATION

- Password hashing: BCrypt or Argon2.
- All traffic over HTTPS.
- Role-Based Access Control (RBAC): gates both the UI and the underlying logic on both clients.
- Anti-tampering: hash verification on critical entities.
- Secure token generation for sessions.
- Input validation on all data sent to the API.
- All actions logged server-side, tagged with the role and platform that performed them.

18. ORDER FLOW (END-TO-END)

1. User adds items to the cart, stored locally on whichever client they are using.
2. Checkout starts.
3. An Order is created through a POST request to the API.
4. The API generates a master order ID.
5. Inventory is updated on the server.
6. The local cart is cleared.
7. The order is added to the user's history on the API.

19. ONLINE-FIRST BEHAVIOR

The system depends on an active connection to the REST API for essentially everything, on both the Android app and the website.

Data Access

When either client opens, it fetches current data from the API, whether that is restaurant listings, menus, or order status, and the UI is populated from that response. Anything that changes data, like placing an order or updating a profile, is sent to the API right away.

Network Dependency

A stable connection is required to use either client. The system checks for connectivity and tells the user when it is missing, for example, "No internet connection. Please check your network settings." If a request fails due to a network issue, the system shows an error and lets the user retry.

Real-Time Updates

The UI updates as responses come back from the API. Where real-time status matters, this can be handled with polling or WebSockets, on either client.

20. SYSTEM INTEGRITY RULES

- Every entity is tied to a user ID.
- No orphan orders are allowed.
- No anonymous transactions are permitted.
- Every action is logged on the API server, regardless of which client or role initiated it.

21. FEASIBILITY ANALYSIS

21.1 Technical Feasibility

Nothing here requires unusual technology: ASP.NET Core 8 with Entity Framework Core on the backend, a native Android client in Kotlin with Retrofit, and a PHP web client. Since both clients consume the same API and enforce the same RBAC rules server-side, extending full role support to both platforms does not add architectural complexity, mainly UI work on each client to present the right screens per role. No special hardware is required.

21.2 Economic Feasibility

The cost is mostly development time. Against that, the system should pay off in operational efficiency and clearer financial tracking.

21.3 Operational Feasibility

Both customers and vendors are already used to mobile and web-based ordering, so training needs should be minimal. Giving vendors and admins the option of either client, instead of locking them into the website, should if anything make onboarding easier.

22. CONCLUSION

Campus Eats is a scalable food ordering and pickup system built around a REST API, with an Android app and a PHP web application that both support all four roles: Student, Standard, Vendor, and Admin. The Fake Restaurant API (AravindAttepu, n.d.) gives it a realistic backend to work against for the purposes of demonstrating the design.

What the system covers:

1. Clearly defined stakeholders.
2. Structured business processes.
3. A full set of functional requirements.
4. A clean separation between backend, mobile client, and web client, with consistent RBAC enforced across both.
5. Online-first operation with REST API integration.
6. A complete RBAC model across four roles, available on both platforms.
7. Real interaction with a live RESTful API.

23. References

A., n.d.. *Fake Restaurant API*. [Online]
Available at: <https://fakerestaurantapi.runasp.net/>
[Accessed 24 July 2026].